

PYTHON PROJECT REPORT

Student Expense & Budget Management System

VITyarthi – Build Your Own Project

Subject	Python Programming
Project Type	Console-based Python Application
Student Name	Naman Tiwari
Registration No.	26BCE10072
College	VIT Bhopal University

Academic Year: 2026–27

1. Introduction

The Student Expense & Budget Management System is a Python-based console application designed to help students record, manage and analyze their daily expenses. The system provides a simple menu-driven interface and stores information using JSON files.

The project demonstrates core Python programming concepts including variables, conditional statements, loops, functions, lists, dictionaries, file handling, exception handling, modular programming and unit testing.

2. Problem Statement

Students often spend money on food, transportation, entertainment and other daily activities without maintaining an organized record. This can make it difficult to understand spending patterns, calculate total expenditure and monitor a monthly budget.

The proposed system provides a structured way to record expenses and obtain basic spending information through a single Python application.

3. Objectives

- Record daily student expenses.
- Provide user registration and login.
- Allow users to add, view and delete expense records.
- Allow users to set and monitor a monthly budget.
- Calculate total spending and category-wise spending.
- Identify the highest spending category.
- Demonstrate modular Python programming and file handling.

4. Functional Requirements

ID	Module	Description
FR1	User Management	Register a new user and authenticate an existing user.
FR2	Expense Management	Add, view and delete expense records.
FR3	Budget Management	Set a monthly budget and compare spending with the budget.
FR4	Reporting	Calculate total expenses and category-wise spending.
FR5	Persistent Storage	Store users, expenses and budgets in a JSON file.

5. Non-Functional Requirements

Usability: The application uses a simple menu-driven interface suitable for beginners.

Reliability: Application data is stored persistently in JSON format.

Maintainability: The system is divided into separate Python modules.

Error Handling: Invalid numeric input and invalid menu selections are handled.

Performance: The application is designed for normal student-scale expense data.

Resource Efficiency: The project uses lightweight local JSON storage without a database server.

6. Technologies Used

- Python 3
- JSON for persistent local storage
- unittest for basic testing
- Visual Studio Code for development
- Git/GitHub for version control and repository management

7. System Architecture

The system follows a modular architecture. The main program receives user input and directs it to the appropriate functional module. The modules communicate with the storage layer, which maintains application data in a JSON file.

```
User
|
v
main.py
|
+--> user.py
+--> expenses.py
+--> budget.py
+--> reports.py
+--> validation.py
|
v
storage.py
|
v
data/expenses.json
```

8. Workflow

```
START → Register/Login → Main Menu
↓
Select Operation
↓
Add / View / Delete Expense
Set / Check Budget
Generate Report
↓
Save / Display Result
↓
Return to Menu
↓
Logout
↓
END
```

9. Use Case Description

Actor: Student

- Register – create an account.
- Login – access the expense management system.
- Add Expense – enter expense name, category and amount.
- View Expenses – display stored expense records.
- Delete Expense – remove an expense by ID.
- Set Budget – define a monthly spending limit.
- Check Budget – compare total spending with the budget.
- Generate Report – view total and category-wise spending.
- Logout – leave the logged-in session.

10. Module Description

File	Purpose
main.py	Controls the application menus and overall workflow.
user.py	Handles registration and login.
expenses.py	Handles adding, viewing and deleting expenses.
budget.py	Handles monthly budget operations.
reports.py	Generates spending summaries and category analysis.
storage.py	Reads and writes JSON application data.
validation.py	Validates usernames, passwords and amounts.
utils.py	Contains reusable formatting utilities.
test_project.py	Contains unit tests for selected functions.

11. Data Storage Design

The application uses a JSON file named **data/expenses.json**. The main data structure contains three sections: users, expenses and budgets.

The expenses section stores records containing an ID, expense title, category, amount and date. This provides persistent storage without requiring a separate database server.

12. Implementation Details

The project uses functions for individual operations and separate files for different responsibilities. Lists and dictionaries are used to store records in memory. JSON serialization is used to save and load the data. Loops and conditional statements control the menu system. Exception handling prevents invalid numeric input from terminating the program.

13. Testing Approach

The project includes a **test_project.py** file using Python's unittest framework. Tests verify username validation, password validation and currency formatting.

The test suite can be executed with: `python -m unittest test_project.py`

Test Case	Input/Condition	Expected Result
TC01	Username: naman	Valid username
TC02	Username: ab	Invalid username
TC03	Password: 1234	Valid password
TC04	Password: 123	Invalid password
TC05	Amount: 250	Rs.250.00 formatting

14. Screenshots / Results

The final report should include screenshots of the running application. Recommended screenshots are:

- Registration screen
- Login screen
- Adding an expense
- Viewing expenses
- Budget check
- Generated expense report

These screenshots can be captured directly from the VS Code terminal after running the application.

15. Challenges Faced

- Designing a simple data structure for multiple users.
- Maintaining persistent data using JSON.
- Handling invalid user input.
- Keeping different features separated into modules.
- Creating tests for reusable functions.

16. Learnings & Key Takeaways

- Modular Python programming
- Functions, lists and dictionaries
- File handling and JSON
- Input validation and exception handling
- Basic data analysis
- Unit testing
- Project structure and documentation

17. Future Enhancements

- Graphical user interface using Tkinter.
- Expense charts using Matplotlib.
- CSV and PDF report export.
- Database integration using SQLite.
- Cloud synchronization.
- More detailed monthly and yearly analytics.

18. References

- Python documentation – Python 3 standard library.
- Python json module documentation.
- Python unittest module documentation.
- VITyarthi Build Your Own Project – General Project Instructions & Submission Guidelines.